

Solution Code - Module 3-Core Java - JavaModules

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q34_JavaModules.java

```
1 | import java.io.File;
2 | import java.io.FileWriter;
3 | import java.io.BufferedReader;
4 | import java.io.InputStreamReader;
5 | import java.nio.file.Files;
6 | import java.nio.file.Path;
7 | import java.nio.file.Paths;
8 | import java.util.Comparator;
9 |
10 | public class Q34_JavaModules {
11 |     public static void main(String[] args) {
12 |         System.out.println("=== Java Modules Compilation & Execution Demonstration ===");
13 |
14 |         String baseDirStr = "temp_modules_demo";
15 |         Path basePath = Paths.get(baseDirStr);
16 |
17 |         try {
18 |             // Clean up any old directories
19 |             if (Files.exists(basePath)) {
20 |                 Files.walk(basePath)
21 |                     .sorted(Comparator.reverseOrder())
22 |                     .map(Path::toFile)
23 |                     .forEach(File::delete);
24 |             }
25 |
26 |             // Create structure for com.utils
27 |             Path utilsSrc = basePath.resolve("com.utils");
28 |             Path utilsPkg = utilsSrc.resolve("com/utils");
29 |             Files.createDirectories(utilsPkg);
30 |
31 |             // Create structure for com.greetings
32 |             Path greetingsSrc = basePath.resolve("com.greetings");
33 |             Path greetingsPkg = greetingsSrc.resolve("com/greetings");
34 |             Files.createDirectories(greetingsPkg);
35 |
36 |             // Create output directory for compiled modules
37 |             Path modsOut = basePath.resolve("mods");
38 |             Files.createDirectories(modsOut);
39 |
40 |             // Write com.utils files
41 |             writeFile(utilsSrc.resolve("module-info.java").toFile(),
42 |                 "module com.utils {\n" +
43 |                 "    exports com.utils;\n" +
```

Solution Code - Module 3-Core Java - JavaModules

```
44 |         "}\n"
45 |     );
46 |     writeFile(utilsPkg.resolve("StringUtils.java").toFile(),
47 |         "package com.utils;\n" +
48 |         "public class StringUtils {\n" +
49 |         "    public static String toTitleCase(String input) {\n" +
50 |         "        if (input == null || input.isEmpty()) return input;\n" +
51 |         "        return input.substring(0, 1).toUpperCase() +
input.substring(1).toLowerCase();\n" +
52 |         "    }\n" +
53 |         "}\n"
54 |     );
55 |
56 |     // Write com.greetings files
57 |     writeFile(greetingsSrc.resolve("module-info.java").toFile(),
58 |         "module com.greetings {\n" +
59 |         "    requires com.utils;\n" +
60 |         "}\n"
61 |     );
62 |     writeFile(greetingsPkg.resolve("Main.java").toFile(),
63 |         "package com.greetings;\n" +
64 |         "import com.utils.StringUtils;\n" +
65 |         "public class Main {\n" +
66 |         "    public static void main(String[] args) {\n" +
67 |         "        String raw = \"welcome to java 9 modules!\";\n" +
68 |         "        System.out.println(\"Greetings Module: Working with com.utils...\");\n" +
69 |         "        System.out.println(\"Original string: \" + raw);\n" +
70 |         "        System.out.println(\"Processed string: \" + StringUtils.toTitleCase(raw));\n" +
71 |         "    }\n" +
72 |         "}\n"
73 |     );
74 |
75 |     System.out.println("1. Created module source directory structure.");
76 |
77 |     // Compile com.utils
78 |     System.out.println("2. Compiling module 'com.utils'...");
79 |     runCommand("javac", "-d", modsOut.resolve("com.utils").toString(),
80 |         utilsSrc.resolve("module-info.java").toString(),
81 |         utilsPkg.resolve("StringUtils.java").toString());
82 |
83 |     // Compile com.greetings
84 |     System.out.println("3. Compiling module 'com.greetings' with module-path...");
85 |     runCommand("javac", "--module-path", modsOut.toString(),
86 |         "-d", modsOut.resolve("com.greetings").toString(),
87 |         greetingsSrc.resolve("module-info.java").toString(),
88 |         greetingsPkg.resolve("Main.java").toString());
89 |
90 |     // Run modular application
91 |     System.out.println("4. Running modular application (java --module-path ... -m
com.greetings/com.greetings.Main):");
```

Solution Code - Module 3-Core Java - JavaModules

```

    92 | |
System.out.println("-----");
    93 | |               runCommand("java", "--module-path", modsOut.toString(), "-m",
"com.greetings/com.greetings.Main");
    94 | |
System.out.println("-----");
    95 |
    96 |         } catch (Exception e) {
    97 |             System.err.println("An error occurred during modular compilation/execution:");
    98 |             e.printStackTrace();
    99 |         } finally {
100 |             // Cleanup directories
101 |             try {
102 |                 if (Files.exists(basePath)) {
103 |                     Files.walk(basePath)
104 |                         .sorted(Comparator.reverseOrder())
105 |                         .map(Path::toFile)
106 |                         .forEach(File::delete);
107 |                 }
108 |                 System.out.println("5. Cleaned up temporary directory.");
109 |             } catch (Exception ignored) {}
110 |         }
111 |     }
112 |
113 |     private static void writeFile(File file, String content) throws Exception {
114 |         try (FileWriter writer = new FileWriter(file)) {
115 |             writer.write(content);
116 |         }
117 |     }
118 |
119 |     private static void runCommand(String... command) throws Exception {
120 |         ProcessBuilder pb = new ProcessBuilder(command);
121 |         pb.redirectErrorStream(true);
122 |         Process p = pb.start();
123 |         try (BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream()))) {
124 |             String line;
125 |             while ((line = reader.readLine()) != null) {
126 |                 System.out.println("    " + line);
127 |             }
128 |         }
129 |         int exitCode = p.waitFor();
130 |         if (exitCode != 0) {
131 |             throw new RuntimeException("Command failed with exit code: " + exitCode);
132 |         }
133 |     }
134 | }
```